Basic (plug search in as a plain tool first) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `search_tool_integration_practice.py`: Doc08's `retrieve()` becomes a real graph tool here, unchanged — the exact wrapper the Build Task's search step reuses. **If not:** the Build Task would be the first place you ever wrapped `retrieve()` as a `@tool`, with no smaller version to trust.

Before you start (once, for every Doc10 exercise): copy Doc08's Build Task files — `chunking.py`, `config.py`, `exceptions.py`, `ingest.py`, `logging_setup.py`, `retriever.py` (Intermediate, Approach 2) — and its `docs/` folder of coffee documents into `10_agent_workflows/practice/`. Then add one small file that builds the knowledge base once, for every exercise to import:

```
# kb.py
# Builds Doc08's knowledge base once; every Doc10 exercise imports it.
from ingest import build_vector_store

collection = build_vector_store("docs")
```

`retrieve(collection, query, k)` returns a list of dicts — `text`, `source_file`, `chunk_index`, `score` — and `score` is a **distance**: lower means closer (Doc08).

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

The graph is Doc09's Real-world graph (`think` → `act` → `think`), with one tool: `search_docs`.

```
# search_tool_integration_practice.py — Basic section
import operator
from typing import Annotated, TypedDict
```

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from kb import collection
from retriever import retrieve


@tool
def search_docs(query):
    """Search the coffee documents for text relevant to the query."""
    results = retrieve(collection, query)
    text = ""
    for result in results:
        text = text + result["text"] + "\n\n"
    return text


class ChatState(TypedDict):
    messages: Annotated[list, operator.add]


def think(state):
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools([search_docs]).invoke(state["messages"])
    return {"messages": [response]}


def act(state):
    tool_messages = []
    for call in state["messages"][-1].tool_calls:
        tool_messages.append({
            "role": "tool",
            "content": search_docs.invoke(call["args"]),
            "tool_call_id": call["id"],
        })
    return {"messages": tool_messages}


def route(state):
    if state["messages"][-1].tool_calls:
        return "act"
    return "finish"


builder = StateGraph(ChatState)
builder.add_node("think", think)
```

```
builder.add_node("act", act)
builder.add_edge(START, "think")
builder.add_conditional_edges("think", route, {"act": "act", "finish": END})
builder.add_edge("act", "think")
graph = builder.compile()

question = "How hot should the water be when brewing?"
result = graph.invoke({"messages": [{"role": "user", "content": question}]})
print(result["messages"][-1].content)
```

Expected output (the wording varies):

```
Water for brewing coffee should be between 195 and 205 degrees Fahrenheit.
```

The model asked for `search_docs`, `act` ran it, and the model answered from the text it got back — the same round trip as Doc06/07, now inside a graph.

This works. It's missing type hints, it returns an empty string when nothing is found, and it has no plan for a search that fails — all fine for a first version.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, a real docstring, and an explicit "no results" message

Replace Basic's `search_docs` with this; the rest of the file stays the same.

```
# search_tool_integration_practice.py — Basic section
@tool
def search_docs(query: str) -> str:
    """Search the company's coffee documents for text relevant to the query.

    Use this whenever the question could be answered by looking something
    up in the coffee documents: brewing, grind size, water, storage.
    """
    results = retrieve(collection, query)
    if not results:
        return "No relevant documents found."
    pieces: list[str] = []
    for result in results:
        pieces.append(result["text"])
    return "\n\n".join(pieces)
```

Difference from Basic: full type hints, and a docstring that tells the model *when* to use the tool — the model decides only from this text (Doc06), so being specific makes it call the tool at the right times. An explicit "no results" message instead of an empty string gives the model a clear signal instead of silence. The pieces are collected in a list and joined once with `"\n\n".join(...)`, instead of growing one string. This version still trusts `retrieve()` never to raise, and returns everything it finds — Approach 2 changes both.

Approach 2 — caught failures and a capped result count

Story: a tool's return value goes straight into the model's context — an uncaught `retrieve()` failure crashes the whole turn, and too many chunks eat the context window on a real document set. **If not:** the first vector-store hiccup in a live run would crash the model turn instead of handing back a sentence the model can react to.

```
# search_tool_integration_practice.py — Basic section
MAX_RESULTS = 2
```

```
@tool
def search_docs(query: str) -> str:
    """Search the company's coffee documents for text relevant to the query.

    Use this whenever the question could be answered by looking something
    up in the coffee documents: brewing, grind size, water, storage.
    """
    try:
        results = retrieve(collection, query, k=MAX_RESULTS)
    except Exception as exc:
        return f"Search is temporarily unavailable: {exc}"

    if not results:
        return "No relevant documents found."
    pieces: list[str] = []
    for result in results:
        pieces.append(result["text"])
    return "\n\n".join(pieces)
```

A failed search now hands the model a short, honest sentence — it can tell the user search is down instead of the run failing. `k=MAX_RESULTS` caps how many chunks one call can add to the context. Like `MAX_ATTEMPTS` in `search_failure`, `MAX_RESULTS` is a plain constant here for readability; in a real project it belongs in `config.py`.

Approach 3 — returning sources separately, for a later approval step

Story: Approach 2's return value is one joined string — fine for the model, but it throws away *which files* were used. `approval_pause` needs a reviewer to see the actual sources. **If not:** the approval step could only ever say "the model searched something," never what it found.

```
# search_tool_integration_practice.py — Basic section
from dataclasses import dataclass

MAX_RESULTS = 2


@dataclass
class SearchResult:
    text: str
    sources: list[str]


def search_docs_with_sources(query: str) -> SearchResult:
    """Search the documents and keep track of which files were used."""
    try:
```

```
        results = retrieve(collection, query, k=MAX_RESULTS)
    except Exception as exc:
        message = f"Search is temporarily unavailable: {exc}"
        return SearchResult(text=message, sources=[])

    if not results:
        return SearchResult(text="No relevant documents found.", sources=[])

    pieces: list[str] = []
    sources: list[str] = []
    for result in results:
        pieces.append(result["text"])
        sources.append(result["source_file"])
    return SearchResult(text="\n\n".join(pieces), sources=sources)


@tool
def search_docs(query: str) -> str:
    """Search the company's coffee documents for text relevant to the query.

    Use this whenever the question could be answered by looking something
    up in the coffee documents: brewing, grind size, water, storage.
    """
    return search_docs_with_sources(query).text


found = search_docs_with_sources("How hot should the water be?")
print("sources:", found.sources)
```

Expected output (first line):

```
sources: ['Water temperature.txt', 'French press basics.txt']
```

`search_docs` (the `@tool`) is still what the model calls and reads. `search_docs_with_sources` is the same logic for a graph **node** to call directly — for example a search node that saves `sources` into state for an approval step to show a human later. `@dataclass` is Doc01's typed holder.

Difference from Approach 1, and between Approaches 2/3: Approach 1's tool works on the happy path, but a failed search crashes the turn and there's no limit on how much text one call returns. Approach 2 fixes both inside the tool. Approach 3 solves a different problem: it keeps the *sources* available to the rest of the graph, for when something downstream — an approval step — must show what was searched.

Which one should you actually write? Approach 2's caught failure and cap belong in every tool that wraps an outside lookup. Approach 3's separate `sources` only once something else in your

graph needs them — which the Build Task's approval step does.